SMARTBRIDGE / SKILL WALLET

Code Architecture, Style Compliance & Structural Metrics

Date: 2 July 2026	**Team ID:** —
Project Name: Caption AI : AI Powered Social Media Caption Generator	

Code-Layout, Readability and Reusability

Well-structured code is essential for maintainability and collaboration. Document the standards, patterns, and principles followed to ensure the codebase is clean, readable, and highly reusable. Provide specific examples of how these practices were implemented in your project.

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
1	**Folder/Directory Structure**	Files are organized logically based on a clean decoupled frontend-backend asset distribution setup. Separation of dynamic logic handlers from static visual rendering styles.	Separate static/css/, static/js/, and templates/ folders under the project root.
2	**Naming Conventions**	Standard PEP 8 rules for backend Python scripts and explicit lowercase semantic naming chains for dashboard styles and rendering assets to keep paths clean.	snake_case for functions in app.py, style.css for presentation definitions.
3	**Code Readability & Comments**	Explicit docstrings and inline programmatic remarks detail input boundary criteria, prompt parameters parsing, and JSON object validation patterns.	Comments mapping JSON schema requirements inside the build_prompt() tool block.
4	**Modularity & Reusability**	Isolating core features into decoupled logical sub-routines. Standard DRY execution architectures enforce reusable asset patterns throughout the app.	Unified API inference blocks utilized inside the main / generate application handler.

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
5	**Formatting & Linting Tools**	Automatic code arrangement, pattern scanning validations, and whitespace uniformity are enforced directly by internal workspace environment formatting rules.	Flake8 and Black parameters integrated into localized VS Code configuration loops.
6	**Error Handling Patterns**	Structured backend response capture wrapping missing tokens or configuration access errors inside explicit HTTP error payloads with safe execution blocks.	Centralized try-except try catch statement blocks isolating JSON exceptions within app.py.